

Intermediate AWS CDK

Instructor – Disha Kamra

Course Objective

- Why CDK & Core Concepts
- Project Structure, Stages (dev/stage/prod), env-aware design
- Constructs deep dive (L2 focus), L1 escape hatches, L3 patterns; importing resources
- Context, lookups, cdk.context.json, feature flags; tagging & standards
- Example app: synth → diff → deploy → review; snapshot tests & troubleshooting
- Pipeline & multi-environment promotion; drift & failure handling; reading generated CloudFormation
- Maintenance: branching strategy, dependency bumps, CDK upgrades, runtime shifts, release notes

Agenda

- **Module 1: Why CDK & Core Concepts (30m)**
 - Why CDK? Pros/cons; alternatives (Terraform, SAM)
 - Core concepts: App → Stacks → Constructs (L1/L2/L3)
- **Module 2: Project Structure & Stages (45m)**
 - CDK project layout; dev/stage/prod
 - Env-aware design; “synth once, deploy many”
 - Stateless vs stateful; stack props; cross-stack outputs
- **Module 3: Constructs Deep Dive + Mini-Lab (75m)**
 - L2 focus: docs, wiring constructs, helpers (e.g., .grantWrite)
 - L1 (Cfn*) + escape hatches; L3 patterns; importing existing resources
 - Mini-lab: add an L2 resource, permission via helper, tweak via escape hatch

Break (15m)

- **Module 4: Context, Lookups & Standards (45m)**
 - Context, lookups, cdk.context.json, feature flags
 - Tagging by stage/stack/construct; org/IaC pre-deploy checks; baselines

Lunch Break (45m)

- **Module 5: Example App Deploy & Troubleshoot (75m)**
 - Walkthrough: synth → diff → deploy → console review
 - Tests: snapshot tests, ignoring S3 asset keys
 - Troubleshooting: synth errors, common cdk diff pitfalls
- **Module 6: Pipeline & Multi-Env (60m)**
 - Enterprise pipeline flow; promote dev→stage→prod
 - Handling drift & deploy failures
 - Reading CDK-generated CloudFormation (mapping CFN ↔ CDK)
- **Module 7: Capstone Lab + Failure/Drift Drills (60m)**
 - Extend app with stage-specific props; introduce a failure → fix → re-deploy
 - Drift detection & remediation; confirm via diff/CFN
- **Module 8: Maintenance & Upgrades + Q&A (30m)**
 - Branching (trunk vs. stable-main), bump deps, CDK upgrades & feature flags
 - Node/Python version shifts; reading release notes; wrap-up

Introducing Yourself 😊

- Name
- What do you do day-to-day?
- What is your experience level with AWS?
- What is your level of experience with IaC?
 - (1 – no experience to 5 – expert)

Module 1:

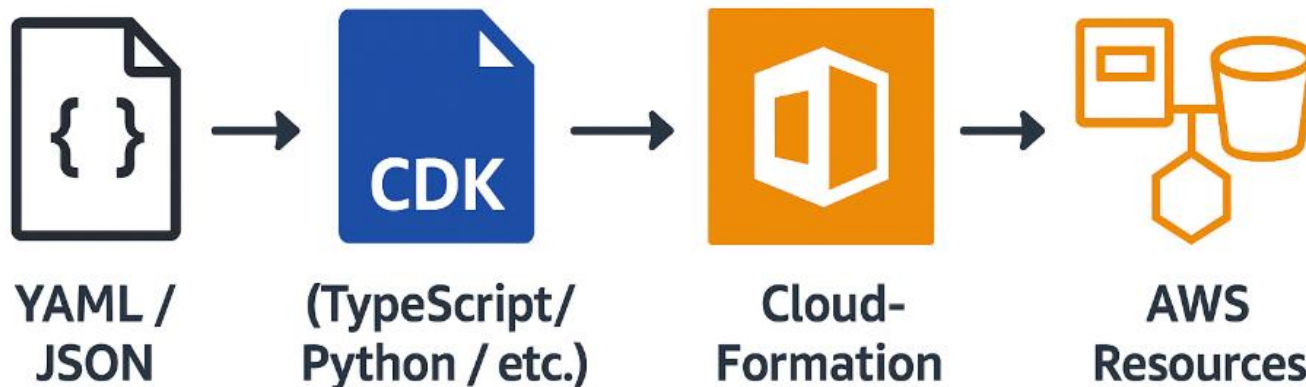
Why CDK & Core Concepts

Learning Agenda

- By the end of this module, learners will:
 - Understand *why* organizations adopt AWS CDK.
 - Compare CDK vs Terraform vs SAM.
 - Explain the CDK building-block hierarchy: **App → Stack → Construct (Level 1–3)**.

Why AWS CDK? Moving from Templates to Real Code

- Write infrastructure using real programming languages
- Faster development: IDE autocomplete, refactoring, types
- Reusable patterns & constructs (no more copy-paste YAML)
- Strong AWS alignment & faster feature adoption
- Still generates CloudFormation under the hood

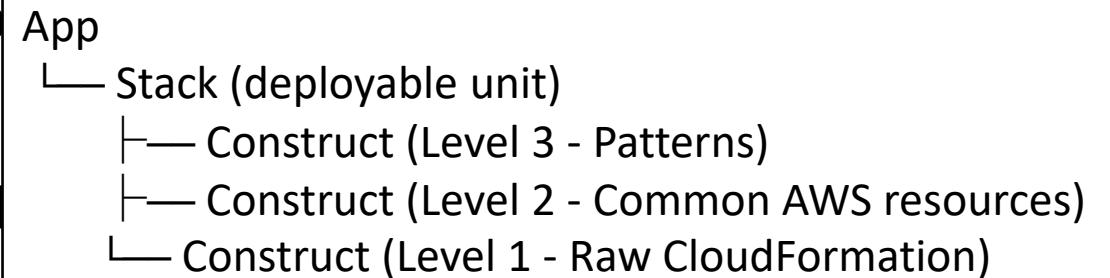


How Does CDK Compare to Other IaC Tools?

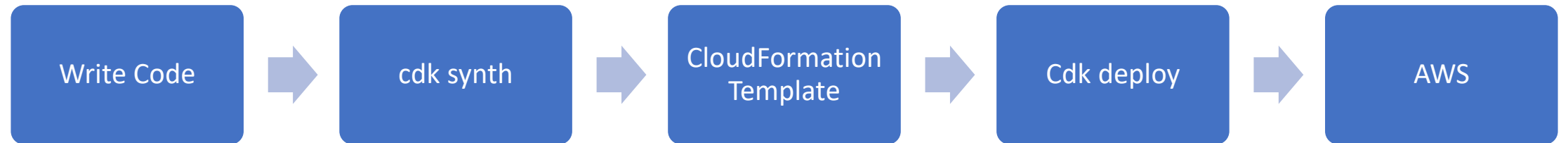
Tool	Language	Strengths	Limitations
Terraform	HCL	Multi-cloud, strong community	Slower AWS updates
SAM	YAML	Serverless optimized	Limited outside Lambda/APIGW
CloudFormation	YAML/JSON	Native AWS	Very verbose
CDK	TypeScript / Python / etc.	Code-first, faster dev, reusable patterns	AWS-only

CDK Building Blocks: The Mental Model

- **App**: The root CDK application
- **Stack**: A deployable CloudFormation stack
- **Construct**: Building block representing AWS resources
 - **L1**: Raw CFN (CfnBucket)
 - **L2**: High-level AWS objects (s3.Bucket)
 - **L3**: Pre-built patterns (LambdaToS3)



How CDK Deploys Infrastructure



Questions?